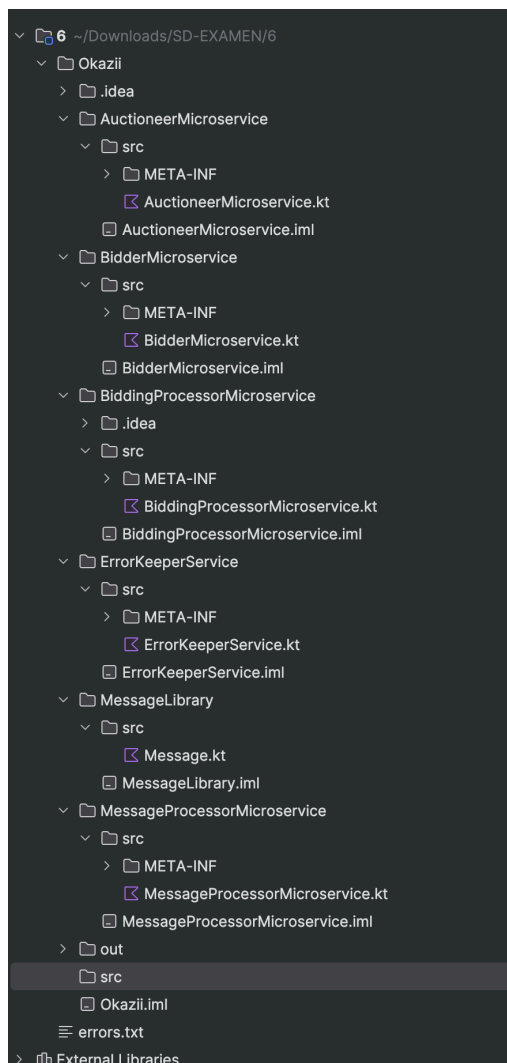


Pornind de la aplicatia exemplu cu Okazii (simulat evident) din laboratorul 7, să se modifice aceasta astfel încât să se introducă un procesor de flux care să realizeze statistici cu privire la erorile si tipul acestora care au apărut (dacă au apărut) pentru o licitatie pana la adjudecare si apoi sa scrie aceste informatii intr-un fisier local. Dacă se consideră necesar se mai pot crea oricâte servicii se doreste conform gândirii studentului. Se vor respecta principiile SOLID pentru microservicii. La prezentare se va arata diagrama de clase si se va explica aceasta impreuna cu gradul de respectare a principiilor. Apoi va urma discutia pe cod.



Notite

1. Creezi fisierul errors.txt ca in poza si il lesi gol
2. Vezi laboratorul pentru detalii de rulare

3. Pentru a putea fi descărcat fișierul a trebuit să șterg artifactele deja create, Vezi lab să le faci la loc

AuctioneerMicroservice

```
import io.reactivex.rxjava3.core.Observable
import io.reactivex.rxjava3.disposables.CompositeDisposable
import io.reactivex.rxjava3.kotlin.subscribeBy
import io.reactivex.rxjava3.plugins.RxJavaPlugins
import java.io.BufferedReader
import java.io.InputStreamReader
import java.net.ServerSocket
import java.net.Socket
import java.net.SocketTimeoutException
import java.util.*
import kotlin.collections.ArrayList
import kotlin.system.exitProcess

class AuctioneerMicroservice {
    private var auctioneerSocket: ServerSocket
    private lateinit var messageProcessorSocket: Socket
    private var receiveBidsObservable: Observable<String>
    private val subscriptions = CompositeDisposable()
    private val bidQueue: Queue<Message> = LinkedList<Message>()
    private val bidderConnections: MutableList<Socket> = mutableListOf()

    companion object Constants {
        const val MESSAGE_PROCESSOR_HOST = "localhost"
        const val MESSAGE_PROCESSOR_PORT = 1600
        const val AUCTIONEER_PORT = 1500
        const val AUCTION_DURATION: Long = 15_000 // licitatia dureaza 15
        secunde
    }

    init {
        auctioneerSocket = ServerSocket(AUCTIONEER_PORT)
        auctioneerSocket.setSoTimeout(AUCTION_DURATION.toInt())
        println("AuctioneerMicroservice se executa pe portul:
        ${auctioneerSocket.localPort}")
        println("Se asteapta oferte de la bidderi...")

        // se creeaza obiectul Observable cu care se genereaza evenimente cand
        se primesc oferte de la bidderi
        receiveBidsObservable = Observable.create<String> { emitter ->
            // se asteapta conexiuni din partea bidderilor
```

```

        while (true) {
            try {
                val bidderConnection = auctioneerSocket.accept()
                bidderConnections.add(bidderConnection)

                // se citeste mesajul de la bidder de pe socketul TCP
                val bufferedReader =
BufferedReader(InputStreamReader(bidderConnection.inputStream))
                val receivedMessage = bufferedReader.readLine()

                // daca se primeste un mesaj gol (NULL), atunci inseamna ca
cealalta parte a socket-ului a fost inchisa
                if (receivedMessage == null) {
                    // deci subscriber-ul respectiv a fost deconectat
                    bufferedReader.close()
                    bidderConnection.close()

                    emitter.onError(Exception("Eroare: Bidder-ul
${bidderConnection.port} a fost deconectat."))
                }

                // se emite ce s-a citit ca si element in fluxul de mesaje
                emitter.onNext(receivedMessage)
            } catch (e: SocketTimeoutException) {
                // daca au trecut cele 15 secunde de la pornirea licitatiei,
inseamna ca licitatiea s-a incheiat
                // se emite semnalul Complete pentru a incheia fluxul de
oferte

                emitter.onComplete()
                break
            }
        }
    }

}

private fun receiveBids() {
    // se incepe prin a primi ofertele de la bidderi
    val receiveBidsSubscription = receiveBidsObservable.subscribeBy(
        onNext = {
            val message = Message.deserialize(it.toByteArray())
            println(message)
            bidQueue.add(message)
        },
        onComplete = {
            // licitatiea s-a incheiat
            // se trimit raspunsurile mai departe catre procesorul de mesaje
            println("Licitatiea s-a incheiat! Se trimit ofertele spre
procesare...")
            forwardBids()
        }
    )
}

```

```

        },
        onError = { println("Eroare: $it") }
    )
    subscriptions.add(receiveBidsSubscription)
}

private fun forwardBids() {
    try {
        messageProcessorSocket = Socket(MESSAGE_PROCESSOR_HOST,
MESSAGE_PROCESSOR_PORT)
        subscriptions.add(Observable.fromIterable(bidQueue).subscribeBy(
            onNext = {
                // trimitere mesaje catre procesorul de mesaje

messageProcessorSocket.getOutputStream().write(it.serialize())
                println("Am trimis mesajul: $it")
            },
            onComplete = {
                println("Am trimis toate ofertele catre MessageProcessor.")
                val bidEndMessage = Message.create(
"$${messageProcessorSocket.localAddress}:${messageProcessorSocket.localPort}",
                "final"
                )

messageProcessorSocket.getOutputStream().write(bidEndMessage.serialize())

                // dupa ce s-a terminat licitatia, se asteapta raspuns de la
MessageProcessorMicroservice
                // cum ca a primit toate mesajele
                val bufferReader =
BufferedReader(InputStreamReader(messageProcessorSocket.inputStream))
                bufferReader.readLine()

                messageProcessorSocket.close()

                finishAuction()
            }
        ))
    } catch (e: Exception) {
        println("Nu ma pot conecta la MessageProcessor!")
        auctioneerSocket.close()
        exitProcess(1)
    }
}

private fun finishAuction() {
    // se asteapta rezultatul licitatiei
    try {

```

```

        val biddingProcessorConnection = auctioneerSocket.accept()
        val bufferedReader =
BufferedReader(InputStreamReader(biddingProcessorConnection.inputStream))

        // se citeste rezultatul licitatiei de la AuctioneerMicroservice de
pe socketul TCP
        val receivedMessage = bufferedReader.readLine()

        val result: Message =
Message.deserialize(receivedMessage.toByteArray())
        val winningPrice = result.body.split(" ")[1].toInt()
        println("Am primit rezultatul licitatiei de la BiddingProcessor:
${result.sender} a castigat cu pretul: $winningPrice")

        // se creeaza mesajele pentru rezultatele licitatiei
        val winningMessage =
Message.create(auctioneerSocket.localSocketAddress.toString(),
                "Licitatie castigata! Pret castigator: $winningPrice")
        val losingMessage =
Message.create(auctioneerSocket.localSocketAddress.toString(),
                "Licitatie pierduta...")

        // se anunta castigatorul
        bidderConnections.forEach {
            if (it.remoteSocketAddress.toString() == result.sender) {
                it.getOutputStream().write(winningMessage.serialize())
            } else {
                it.getOutputStream().write(losingMessage.serialize())
            }
            it.close()
        }
    } catch (e: Exception) {
        println("Nu ma pot conecta la BiddingProcessor!")
        auctioneerSocket.close()
        exitProcess(1)
    }
}

// se elibereaza memoria din multimea de Subscriptions
subscriptions.dispose()
}

fun run() {
    receiveBids()
}

}

fun main(args: Array<String>) {
    val bidderMicroservice = AuctioneerMicroservice()
    bidderMicroservice.run()
}

```

```
}
```

BidderMicroservice

```
import io.reactivex.rxjava3.core.Observable
import io.reactivex.rxjava3.kotlin.subscribeBy
import java.io.BufferedReader
import java.io.InputStreamReader
import java.net.Socket
import kotlin.Exception
import kotlin.random.Random
import kotlin.system.exitProcess

class BidderMicroservice {
    private var auctioneerSocket: Socket
    private var auctionResultObservable: Observable<String>
    private var myIdentity: String = "[BIDDER_NECONECTAT]"

    companion object Constants {
        const val AUCTIONEER_HOST = "localhost"
        const val AUCTIONEER_PORT = 1500
        const val MAX_BID = 10_000
        const val MIN_BID = 1_000
    }

    init {
        try {
            auctioneerSocket = Socket(AUCTIONEER_HOST, AUCTIONEER_PORT)
            println("M-am conectat la Auctioneer!")

            myIdentity = "[$${auctioneerSocket.localPort}]"

            // se creeaza un obiect Observable ce va emite mesaje primite
            // printr-un TCP
            // fiecare mesaj primit reprezinta un element al fluxului de date
            // reactiv
            auctionResultObservable = Observable.create<String> { emitter ->
                // se citeste raspunsul de pe socketul TCP
                val bufferedReader =
                    BufferedReader(InputStreamReader(auctioneerSocket.inputStream))
                val receivedMessage = bufferedReader.readLine()

                // daca se primeste un mesaj gol (NULL), atunci inseamna ca
                // cealalta parte a socket-ului a fost inchisa
                if (receivedMessage == null) {
                    bufferedReader.close()
                    auctioneerSocket.close()
                }
            }
        } catch (e: Exception) {
            e.printStackTrace()
            exitProcess(1)
        }
    }
}
```

```

        emitter.onError(Exception("AuctioneerMicroservice s-a
deconectat."))
        return@create
    }

    // mesajul primit este emis in flux
    emitter.onNext(receivedMessage)

    // deoarece se asteapta un singur mesaj, in continuare se emite
semnalul de incheiere al fluxului
    emitter.onComplete()

    bufferReader.close()
    auctioneerSocket.close()
}
} catch (e: Exception) {
    println("$myIdentity Nu ma pot conecta la Auctioneer!")
    exitProcess(1)
}
}

private fun bid() {
    // se genereaza o oferta aleatorie din partea bidderului curent
    val pret = Random.nextInt(MIN_BID, MAX_BID)

    // se creeaza mesajul care incapsuleaza oferta
    val biddingMessage =
Message.create("${auctioneerSocket.localAddress}:${auctioneerSocket.localPort}"
,
        "licitez $pret")

    // bidder-ul trimite pretul pentru care doreste sa liciteze
    val serializedMessage = biddingMessage.serialize()
    auctioneerSocket.getOutputStream().write(serializedMessage)

    // exista o sansa din 2 ca bidder-ul sa-si trimita oferta de 2 ori,
eronat
    if (Random.nextBoolean()) {
        auctioneerSocket.getOutputStream().write(serializedMessage)
    }
}

private fun waitForResult() {
    println("$myIdentity Astept rezultatul licitatiei...")
    // bidder-ul se inscrie pentru primirea unui raspuns la oferta trimisa
de acesta
    val auctionResultSubscription = auctionResultObservable.subscribeBy(

```

```

        // cand se primeste un mesaj in flux, inseamna ca a sosit rezultatul
licitatiei
        onNext = {
            val resultMessage: Message =
Message.deserialize(it.toByteArray())
            println("$myIdentity Rezultat licitatie: ${resultMessage.body}")
        },
        onError = {
            println("$myIdentity Eroare: $it")
        }
    )

    // se elibereaza memoria obiectului Subscription
    auctionResultSubscription.dispose()
}

fun run() {
    bid()
    waitForResult()
}
}

fun main(args: Array<String>) {
    val bidderMicroservice = BidderMicroservice()
    bidderMicroservice.run()
}

```

BiddingProcessorMicroservice

```

import io.reactivex.rxjava3.core.Observable
import io.reactivex.rxjava3.disposables.CompositeDisposable
import io.reactivex.rxjava3.kotlin.subscribeBy
import java.io.BufferedReader
import java.io.InputStreamReader
import java.net.ServerSocket
import java.net.Socket
import java.util.*
import kotlin.system.exitProcess

class BiddingProcessorMicroservice {
    private var biddingProcessorSocket: ServerSocket
    private lateinit var auctioneerSocket: Socket
    private var receiveProcessedBidsObservable: Observable<String>
    private val subscriptions = CompositeDisposable()
    private val processedBidsQueue: Queue<Message> = LinkedList<Message>()

    companion object Constants {

```



```

const val BIDDING_PROCESSOR_PORT = 1700
const val AUCTIONEER_PORT = 1500
const val AUCTIONEER_HOST = "localhost"
}

init {
    biddingProcessorSocket = ServerSocket(BIDDING_PROCESSOR_PORT)
    println("BiddingProcessorMicroservice se executa pe portul:
${biddingProcessorSocket.localPort}")
    println("Se asteapta ofertele pentru finalizarea licitatiei...")

    // se asteapta mesaje primite de la MessageProcessorMicroservice
    val messageProcessorConnection = biddingProcessorSocket.accept()
    val bufferedReader =
BufferedReader(InputStreamReader(messageProcessorConnection.inputStream))

    // se creeaza obiectul Observable cu care se captureaza mesajele de la
MessageProcessorMicroservice
    receiveProcessedBidsObservable = Observable.create<String> { emitter ->
        while (true) {
            // se citeste mesajul de la MessageProcessorMicroservice de pe
socketul TCP
            val receivedMessage = bufferedReader.readLine()

            // daca se primeste un mesaj gol (NULL), atunci inseamna ca
cealalta parte a socket-ului a fost inchisa
            if (receivedMessage == null) {
                // deci MessageProcessorMicroservice a fost deconectat
                bufferedReader.close()
                messageProcessorConnection.close()

                emitter.onError(Exception("Eroare:
MessageProcessorMicroservice ${messageProcessorConnection.port} a fost
deconectat."))
                break
            }

            // daca mesajul este cel de tip „FINAL DE LISTA DE MESAJE”
(avand corpul "final"), atunci se emite semnalul Complete
            if (Message.deserialize(receivedMessage.toByteArray()).body ==
"final") {
                emitter.onComplete()

                // s-au primit toate mesajele de la
MessageProcessorMicroservice, i se trimite un mesaj pentru a semnala
                // acest lucru
                val finishedBidsMessage = Message.create(

```

```

"${messageProcessorConnection.localAddress}:${messageProcessorConnection.localPort}",
        "am primit tot"
    )

messageProcessorConnection.getOutputStream().write(finishedBidsMessage.serialize())

        messageProcessorConnection.close()

        break
    } else {
        // se emite ce s-a citit ca si element in fluxul de mesaje
        emitter.onNext(receivedMessage)
    }
}
}

private fun receiveProcessedBids() {
    // se primesc si se adauga in coada ofertele procesate de la MessageProcessorMicroservice
    val receiveProcessedBidsSubscription = receiveProcessedBidsObservable
        .subscribeBy(
            onNext = {
                val message = Message.deserialize(it.toByteArray())
                println(message)
                processedBidsQueue.add(message)
            },
            onComplete = {
                // s-a incheiat primirea tuturor mesajelor
                // se decide castigatorul licitatiei
                decideAuctionWinner()
            },
            onError = { println("Eroare: $it") }
        )
    subscriptions.add(receiveProcessedBidsSubscription)
}

private fun decideAuctionWinner() {
    // se calculeaza castigatorul ca fiind cel care a ofertat cel mai mult
    val winner: Message? = processedBidsQueue.toList().maxBy {
        // corpul mesajului e de forma "licitez <SUMA_LICITATA>"
        // se preia a doua parte, separata de spatiu
        it.body.split(" ")[1].toInt()
    }

    println("Castigatorul este: ${winner?.sender}")
}

```

```

        try {
            auctioneerSocket = Socket(AUCTIONEER_HOST, AUCTIONEER_PORT)

            // se trimite castigatorul catre AuctioneerMicroservice
            auctioneerSocket.getOutputStream().write(winner!!.serialize())
            auctioneerSocket.close()

            println("Am anuntat castigatorul catre AuctioneerMicroservice.")
        } catch (e: Exception) {
            println("Nu ma pot conecta la Auctioneer!")
            biddingProcessorSocket.close()
            exitProcess(1)
        }
    }

    fun run() {
        receiveProcessedBids()

        // se elibereaza memoria din multimea de Subscriptions
        subscriptions.dispose()
    }
}

fun main(args: Array<String>) {
    val biddingProcessorMicroservice = BiddingProcessorMicroservice()
    biddingProcessorMicroservice.run()
}

```

ErrorKeeperService

```

import io.reactivex.rxjava3.core.Observable
import io.reactivex.rxjava3.disposables.CompositeDisposable
import io.reactivex.rxjava3.kotlin.subscribeBy
import java.io.BufferedReader
import java.io.InputStreamReader
import java.net.ServerSocket
import java.net.Socket
import java.util.*
import kotlin.system.exitProcess

class BiddingProcessorMicroservice {
    private var biddingProcessorSocket: ServerSocket
    private lateinit var auctioneerSocket: Socket
    private var receiveProcessedBidsObservable: Observable<String>
    private val subscriptions = CompositeDisposable()
    private val processedBidsQueue: Queue<Message> = LinkedList<Message>()
}

```

```

companion object Constants {
    const val BIDDING_PROCESSOR_PORT = 1700
    const val AUCTIONEER_PORT = 1500
    const val AUCTIONEER_HOST = "localhost"
}

init {
    biddingProcessorSocket = ServerSocket(BIDDING_PROCESSOR_PORT)
    println("BiddingProcessorMicroservice se executa pe portul:
${biddingProcessorSocket.localPort}")
    println("Se asteapta ofertele pentru finalizarea licitatiei...")

    // se asteapta mesaje primite de la MessageProcessorMicroservice
    val messageProcessorConnection = biddingProcessorSocket.accept()
    val bufferedReader =
BufferedReader(InputStreamReader(messageProcessorConnection.inputStream))

    // se creeaza obiectul Observable cu care se captureaza mesajele de la
MessageProcessorMicroservice
    receiveProcessedBidsObservable = Observable.create<String> { emitter ->
        while (true) {
            // se citeste mesajul de la MessageProcessorMicroservice de pe
socketul TCP

            val receivedMessage = bufferedReader.readLine()

            // daca se primeste un mesaj gol (NULL), atunci inseamna ca
cealalta parte a socket-ului a fost inchisa
            if (receivedMessage == null) {
                // deci MessageProcessorMicroservice a fost deconectat
                bufferedReader.close()
                messageProcessorConnection.close()

                emitter.onError(Exception("Eroare:
MessageProcessorMicroservice ${messageProcessorConnection.port} a fost
deconectat."))

                break
            }

            // daca mesajul este cel de tip „FINAL DE LISTA DE MESAJE”
(avand corpul "final"), atunci se emite semnalul Complete
            if (Message.deserialize(receivedMessage.toByteArray()).body ==
"final") {

                emitter.onComplete()

                // s-au primit toate mesajele de la
MessageProcessorMicroservice, i se trimite un mesaj pentru a semnala
                // acest lucru
                val finishedBidsMessage = Message.create(

```

```

"${messageProcessorConnection.localAddress}:${messageProcessorConnection.localPort}",
        "am primit tot"
    )

messageProcessorConnection.getOutputStream().write(finishedBidsMessage.serialize())

        messageProcessorConnection.close()

        break
    } else {
        // se emite ce s-a citit ca si element in fluxul de mesaje
        emitter.onNext(receivedMessage)
    }
}
}

private fun receiveProcessedBids() {
    // se primesc si se adauga in coada ofertele procesate de la MessageProcessorMicroservice
    val receiveProcessedBidsSubscription = receiveProcessedBidsObservable
        .subscribeBy(
            onNext = {
                val message = Message.deserialize(it.toByteArray())
                println(message)
                processedBidsQueue.add(message)
            },
            onComplete = {
                // s-a incheiat primirea tuturor mesajelor
                // se decide castigatorul licitatiei
                decideAuctionWinner()
            },
            onError = { println("Eroare: $it") }
        )
    subscriptions.add(receiveProcessedBidsSubscription)
}

private fun decideAuctionWinner() {
    // se calculeaza castigatorul ca fiind cel care a ofertat cel mai mult
    val winner: Message? = processedBidsQueue.toList().maxBy {
        // corpul mesajului e de forma "licitez <SUMA_LICITATA>"
        // se preia a doua parte, separata de spatiu
        it.body.split(" ")[1].toInt()
    }

    println("Castigatorul este: ${winner?.sender}")
}

```

```

    try {
        auctioneerSocket = Socket(AUCTIONEER_HOST, AUCTIONEER_PORT)

        // se trimite castigatorul catre AuctioneerMicroservice
        auctioneerSocket.getOutputStream().write(winner!!.serialize())
        auctioneerSocket.close()

        println("Am anuntat castigatorul catre AuctioneerMicroservice.")
    } catch (e: Exception) {
        println("Nu ma pot conecta la Auctioneer!")
        biddingProcessorSocket.close()
        exitProcess(1)
    }
}

fun run() {
    receiveProcessedBids()

    // se elibereaza memoria din multimea de Subscriptions
    subscriptions.dispose()
}

fun main(args: Array<String>) {
    val biddingProcessorMicroservice = BiddingProcessorMicroservice()
    biddingProcessorMicroservice.run()
}

```

[Message.kt](#)

```

import java.text.SimpleDateFormat
import java.util.*

class Message private constructor(val sender: String, val body: String, val
timestamp: Date) {
    companion object {
        fun create(sender: String, body: String): Message {
            return Message(sender, body, Date())
        }

        fun deserialize(msg: ByteArray): Message {
            val msgString = String(msg)
            val (timestamp, sender, body) = msgString.split(' ', limit = 3)

            return Message(sender, body, Date(timestamp.toLong()))
        }
    }
}

```

```

    fun serialize(): ByteArray {
        return "${timestamp.time} $sender $body\n".toByteArray()
    }

    override fun toString(): String {
        val dateString = SimpleDateFormat("dd-MM-yyyy
HH:mm:ss").format(timestamp)
        return "[$dateString] $sender >>> $body"
    }
}

fun main(args: Array<String>) {
    val msg = Message.create("localhost:4848", "test mesaj")
    println(msg)
    val serialized = msg.serialize()
    val deserialized = Message.deserialize(serialized)
    println(deserialized)
}

```

MessageProcessorMicroservice

```

import io.reactivex.rxjava3.core.Observable
import io.reactivex.rxjava3.disposables.CompositeDisposable
import io.reactivex.rxjava3.kotlin.subscribeBy
import java.io.BufferedReader
import java.io.InputStreamReader
import java.net.ServerSocket
import java.net.Socket
import java.util.*
import kotlin.system.exitProcess

class MessageProcessorMicroservice {
    private var messageProcessorSocket: ServerSocket
    private lateinit var biddingProcessorSocket: Socket
    private var auctioneerConnection: Socket
    private var receiveInQueueObservable: Observable<String>
    private val subscriptions = CompositeDisposable()
    private val messageQueue: Queue<Message> = LinkedList<Message>()

    // ERROR KEEPER
    private lateinit var errorKeeperSocket: Socket

    companion object Constants {
        const val MESSAGE_PROCESSOR_PORT = 1600
        const val BIDDING_PROCESSOR_HOST = "localhost"
        const val BIDDING_PROCESSOR_PORT = 1700
    }
}

```

```

        // ERROR KEEPER
        const val ERROR_KEEPER_HOST = "localhost"
        const val ERROR_KEEPER_PORT = 1750
    }

    init {
        messageProcessorSocket = ServerSocket(MESSAGE_PROCESSOR_PORT)
        println("MessageProcessorMicroservice se executa pe portul:
${messageProcessorSocket.localPort}")
        println("Se asteapta mesaje pentru procesare...")

        // se asteapta mesaje primite de la AuctioneerMicroservice
        auctioneerConnection = messageProcessorSocket.accept()
        val bufferedReader =
BufferedReader(InputStreamReader(auctioneerConnection.inputStream))

        // se creeaza obiectul Observable cu care se captureaza mesajele de la
AuctioneerMicroservice
        receiveInQueueObservable = Observable.create<String> { emitter ->
            while (true) {
                // se citeste mesajul de la AuctioneerMicroservice de pe
socketul TCP
                val receivedMessage = bufferedReader.readLine()

                // daca se primeste un mesaj gol (NULL), atunci inseamna ca
cealalta parte a socket-ului a fost inchisa
                if (receivedMessage == null) {
                    // deci subscriber-ul respectiv a fost deconectat
                    bufferedReader.close()
                    auctioneerConnection.close()

                    emitter.onError(Exception("Eroare: AuctioneerMicroservice
${auctioneerConnection.port} a fost deconectat."))
                    break
                }

                // daca mesajul este cel de incheiere a licitatiei (avand corpul
"final"), atunci se emite semnalul Complete
                if (Message.deserialize(receivedMessage.toByteArray()).body ==
"final") {
                    emitter.onComplete()

                    break
                } else {
                    // se emite ce s-a citit ca si element in fluxul de mesaje
                    emitter.onNext(receivedMessage)
                }
            }
        }
    }
}

```



```

    }

    private fun receiveAndProcessMessages() {
        //ERROR KEEPER
        errorKeeperSocket = Socket(ERROR_KEEPER_HOST, ERROR_KEEPER_PORT)

        // se primesc si se adauga in coada mesajele de la
AuctioneerMicroservice
        val receiveInQueueSubscription = receiveInQueueObservable
        ///TODO --- filtrati duplicatele folosind operatorul de filtrare
        .subscribeBy(
            onNext = {
                val message = Message.deserialize(it.toByteArray())
                println(message)
                messageQueue.add(message)
            },
            onComplete = {
                // s-a incheiat primirea tuturor mesajelor
                ///TODO --- se ordoneaza in functie de data si ora cand
mesajele au fost primite

                // s-au primit toate mesajele de la AuctioneerMicroservice,
i se trimite un mesaj pentru a semnala
                // acest lucru
                val finishedMessagesMessage = Message.create(
                    "${auctioneerConnection.localAddress}:${auctioneerConnection.localPort}",
                    "am primit tot"
                )

                auctioneerConnection.getOutputStream().write(finishedMessagesMessage.serialize(
                ))

                auctioneerConnection.close()

                // se trimit mai departe mesajele procesate catre
BiddingProcessor

                sendProcessedMessages()
            },
            onError = {
                println("Eroare: $it")

                errorKeeperSocket.getOutputStream().write(it.toString().toByteArray())
                errorKeeperSocket.close()
            }
        )

        subscriptions.add(receiveInQueueSubscription)
    }

    private fun sendProcessedMessages() {

```

```

        try {
            biddingProcessorSocket = Socket(BIDDING_PROCESSOR_HOST,
BIDDING_PROCESSOR_PORT)

            println("Trimit urmatoarele mesaje:")
            Observable.fromIterable(messageQueue).subscribeBy(
                onNext = {
                    println(it.toString())

                    // trimitere mesaje catre procesorul licitatiei, care decide
rezultatul final

                    biddingProcessorSocket.getOutputStream().write(it.serialize())
                },
                onComplete = {
                    val noMoreMessages = Message.create(
"$${biddingProcessorSocket.localAddress}:${biddingProcessorSocket.localPort}",
                    "final"
                )

                    biddingProcessorSocket.getOutputStream().write(noMoreMessages.serialize())
                    biddingProcessorSocket.close()

                    // se elibereaza memoria din multimea de Subscriptions
                    subscriptions.dispose()
                }
            )
        } catch (e: Exception) {
            println("Nu ma pot conecta la BiddingProcessor!")
            messageProcessorSocket.close()
            exitProcess(1)
        }
    }

    fun run() {
        receiveAndProcessMessages()
    }
}

fun main(args: Array<String>) {
    val messageProcessorMicroservice = MessageProcessorMicroservice()
    messageProcessorMicroservice.run()
}

```